Model selection and performance evaluation

- Generalization and unseen data
- Training, validation and test sets
- Cross-validation and model selection
- Hyperparameter tuning
- Performance metrics
- A complete evaluation workflow

Learning from data

Suppose we have **10,000 labelled images** of cats and dogs:

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}, \qquad y_i \in \{0, 1\}.$$

We learn a predictive function $f_{\boldsymbol{\theta}}(\mathbf{x})$ by minimizing empirical loss:

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^{N} L(f_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i).$$

Question: Will this model work on images it has never seen?

Training error and generalization error

Training error is measured on the examples used to learn the model:

$$\hat{R}_{\text{train}}(f) = \frac{1}{N} \sum_{i=1}^{N} L(f(\mathbf{x}_i), y_i).$$

Generalization error is the expected loss on fresh examples:

$$R(f) = \mathbb{E}_{(\mathbf{X},Y)\sim P_{\text{data}}}[L(f(\mathbf{X}), Y)].$$

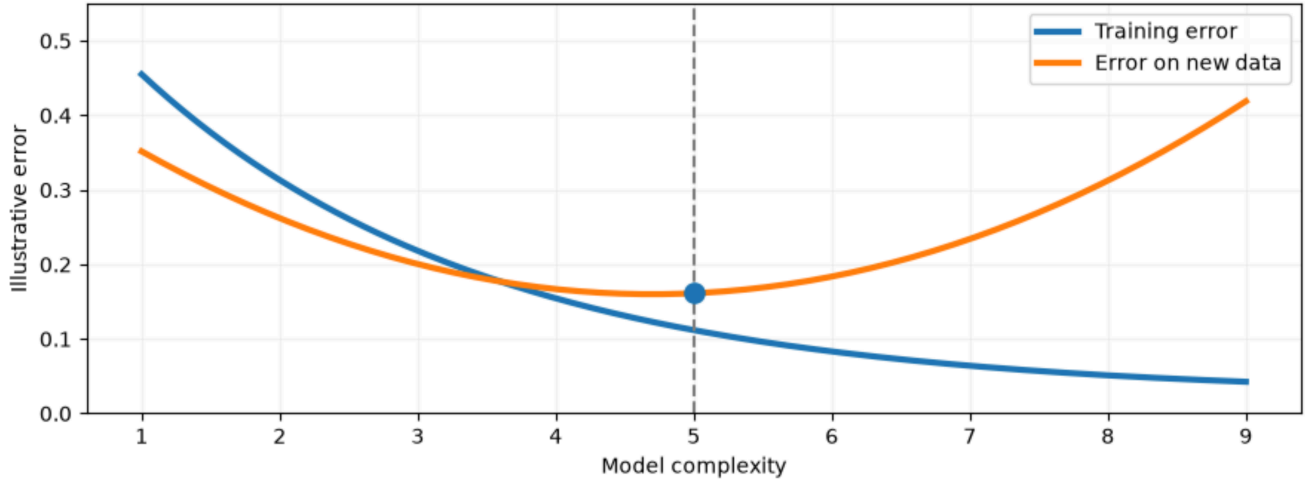
We know the training data, but not the complete distribution P_{data}.

- Model A: 99% training accuracy, 72% on unseen data.
- Model B: 91% training accuracy, 88% on unseen data.

Which model would you choose?

Overfitting: more complex is not always better

Model complexity 5



Training error can continue to decrease even when the error on unseen data increases.

Estimating performance on new data

Evaluate a **fixed model** on M new, independent examples:

$$\hat{R}_{\text{new}}(f) = \frac{1}{M} \sum_{i=1}^{M} L(f(\mathbf{x}_i), y_i).$$

If the evaluation examples are i.i.d. from the target distribution:

$$\mathbb{E}[\hat{R}_{\text{new}}(f) \mid f] = R(f).$$

- The estimate contains statistical uncertainty.
- A larger evaluation sample usually makes it more stable.
- Repeatedly using the test set to choose a model destroys independence.

Why do we need three sets?

- **Training:** learn model parameters.
- **Validation:** choose architecture, hyperparameters, and checkpoints.
- **Test:** evaluate the final chosen model on untouched data.

Dataset	Updates model weights?	Guides model selection?
Training	Yes	Yes
Validation	Usually not directly	Yes
Test	No	**No**

A test set used repeatedly for tuning is no longer an independent final test.

Train–validation–test split

Training (%) 70

Validation (%) 15



For 10,000 examples: **7,000 train**, **1,500 validation**, **1,500 test**. The percentages are illustrative.

Parameters and hyperparameters

Training adjusts **parameters** such as neural-network weights:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} \hat{R}_{\text{train}}(\boldsymbol{\theta}_t).$$

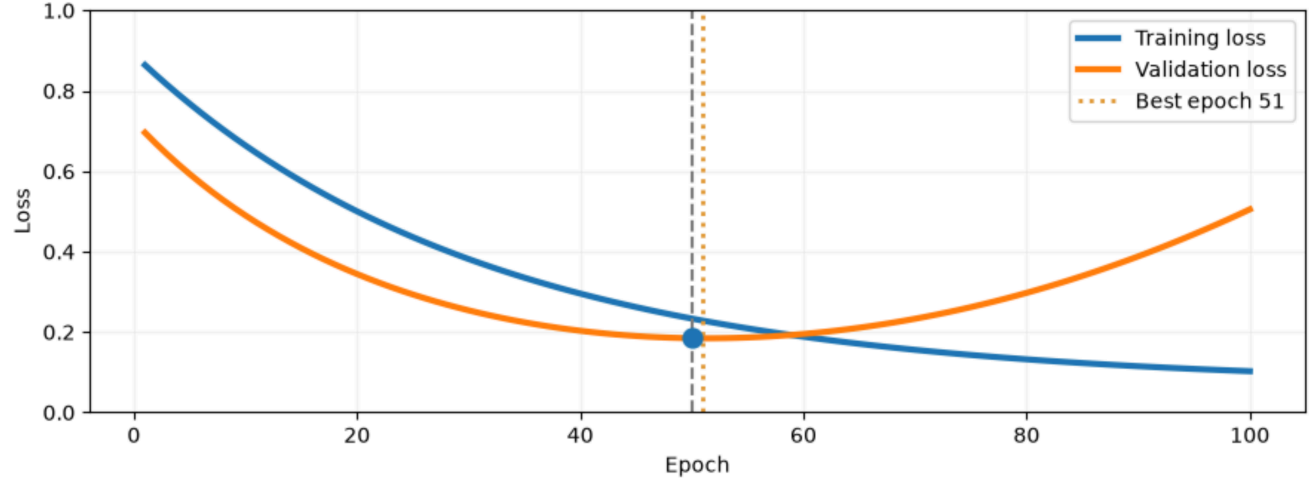
Validation helps choose **hyperparameters**, such as learning rate η:

Learning rate	Validation loss
0.1	0.52
0.01	**0.31**
0.001	0.43

We choose $\eta = 0.01$, but **0.31 is not an independent final evaluation**.

Early stopping

Training epoch 50



Save the checkpoint with the lowest validation loss. With *patience = 10*, stop after ten epochs without improvement and restore the best weights.

Why not tune on the test set?

If we compare many learning rates, architectures, or checkpoints using the test set, we gradually adapt our choices to its random fluctuations.

- **Training** learns weights.
- **Validation** influences the chosen model, even without contributing gradients.
- **Test** is accessed only after decisions are fixed.

A test accuracy of 87% estimates future performance. It does **not** guarantee 87% accuracy on every future dataset.

The limitation of a single validation split

With only **500 development examples**, the validation score depends on the particular split.

- One split may give 84% accuracy.
- Another split may give 90% accuracy.

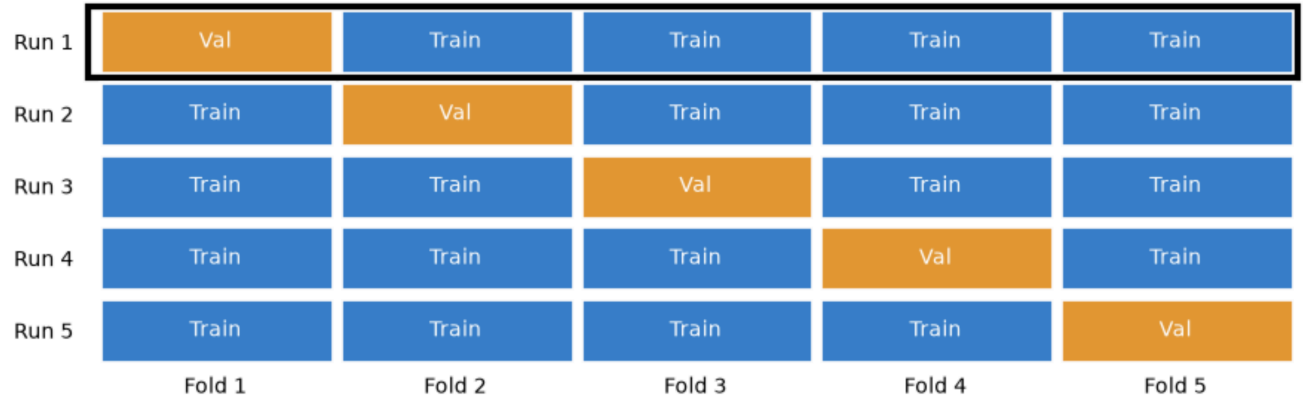
K-fold cross-validation rotates the validation subset:

1. Divide development data into K folds.
2. Train a fresh model on $K - 1$ folds.
3. Validate on the remaining fold.
4. Repeat until every fold has served as validation.

The independent test set stays outside this process.

Five-fold cross-validation

Validation fold 1



Run 1: 400 examples for training, 100 for validation. Each run starts with a new model; weights are not carried over.

Which of the five models do we keep?

Fold model	Validation accuracy
f_{θ_1}	86%
f_{θ_2}	90%
f_{θ_3}	88%
f_{θ_4}	**91%**
f_{θ_5}	85%

$$\text{Accuracy}_{\text{CV}} = \frac{0.86 + 0.90 + 0.88 + 0.91 + 0.85}{5} = 0.88.$$

Usually, none. Cross-validation evaluates a *training procedure*, not which set of fitted weights happened to score highest on its fold.

Refit: build the final model

1. Evaluate candidate configurations with cross-validation.
2. Choose the configuration λ^* with the best mean validation score.
3. Train a **new model** using all development examples.
4. Evaluate that model on the held-out test set.

$$\boldsymbol{\theta}_{\text{final}} = \text{Train}(\mathcal{D}_{\text{development}}, \lambda^*).$$

The five cross-validation models are temporary. Retaining them and averaging their predictions would be **ensemble learning**, a different approach.

Choosing the right cross-validation strategy

Strategy	Use case
Stratified K-fold	Approximate class proportions in each fold
Group K-fold	Keep related examples (e.g. one patient) in the same group
Time-series split	Respect time order; do not train on the future

Early stopping: use an inner validation subset if the outer validation fold is intended as an unbiased evaluation fold.

$K = 5$ and $K = 10$ are common choices. More folds generally mean more training runs.

Hyperparameter selection

Recall: Support Vector Machines (SVMs). An SVM finds a decision boundary that separates classes with a large margin. Some training points, called *support vectors*, determine that boundary.

With the **radial basis function (RBF) kernel**, the boundary can be non-linear. The kernel measures similarity between two feature vectors:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2\right).$$

- C **(regularization):** small C allows more margin violations and favors a wider, smoother margin; large C penalizes violations more strongly and tries to fit training data more closely.
- γ **(kernel width):** small γ means each point influences a wider region, usually giving a smoother boundary; large γ means more localized influence and potentially a more complex boundary.

C and γ are **hyperparameters**, not learned SVM coefficients. We select them using cross-validation.

Grid search with five-fold CV

C 10 Gamma 0.01



Selected configuration: mean CV accuracy 91%. There are **12 configurations × 5 folds = 60 temporary fits**, followed by a final refit.

Grid search versus random search

Grid search evaluates every combination in a specified grid. With five hyperparameters and ten values each, there are 10^5 configurations.

Random search samples a fixed budget of combinations, for example 30:

$$C \sim \mathrm{LogUniform}(10^{-3}, 10^{3}), \qquad \gamma \sim \mathrm{LogUniform}(10^{-4}, 10^{1}).$$

The selected configuration is:

$$\lambda^{*} = \arg\max_{\lambda \in \Lambda} \mathrm{Accuracy}_{\mathrm{CV}}(\lambda).$$

A large search space often makes random search attractive.

Data leakage

Consider standardization:

$$z_j = \frac{x_j - \mu_j}{\sigma_j}.$$

Incorrect: compute μ_j and σ_j on the full dataset before cross-validation.

Correct: estimate preprocessing on the **training part of each fold** and apply it to that fold's validation part.

The confusion matrix

Suppose the positive class means *patient with the condition*. On 1,000 test cases:

Actual ↓ / Predicted →	Positive	Negative
Positive	**TP = 80**	**FN = 20**
Negative	**FP = 90**	**TN = 810**

- **True positive:** a positive case correctly detected.
- **False negative:** a positive case missed.
- **False positive:** a false alarm.
- **True negative:** a negative case correctly identified.

Accuracy can be misleading

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{890}{1000} = 0.89.$$

But if only 100 of the 1,000 patients are positive, a classifier that **always predicts negative** achieves **90% accuracy** and detects no positive cases.

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{80}{170} \approx 0.471,$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{80}{100} = 0.80,$$

$$\text{Specificity} = \frac{TN}{TN + FP} = \frac{810}{900} = 0.90.$$

F1-score and balanced accuracy

$$F_1 = 2\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN} = \frac{160}{270} \approx 0.593.$$

$$\text{BalancedAccuracy} = \frac{\text{Sensitivity} + \text{Specificity}}{2} = \frac{0.8 + 0.9}{2} = 0.85.$$

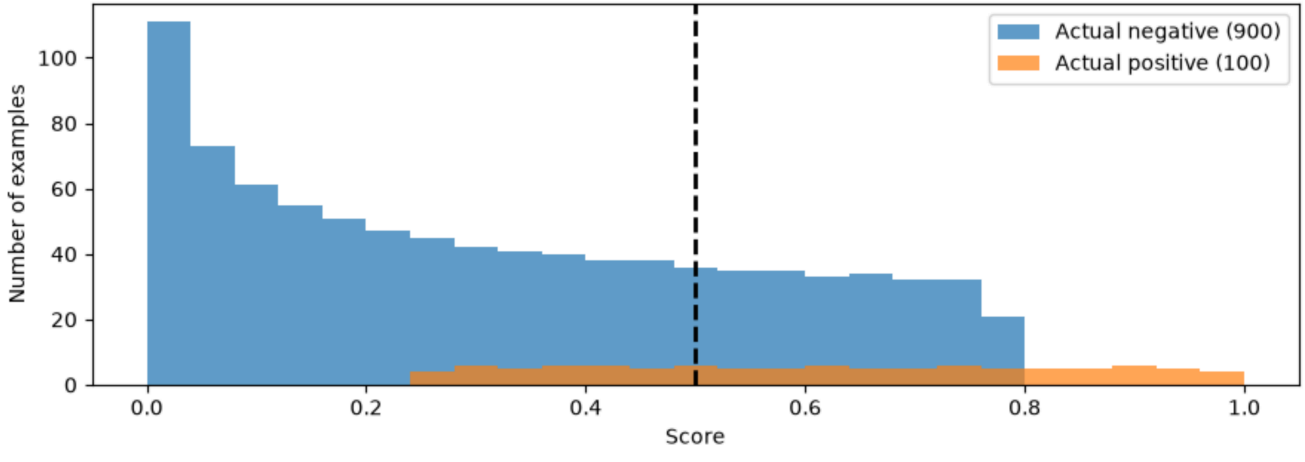
More generally:

$$F_\beta = (1 + \beta^2)\frac{\text{Precision} \cdot \text{Recall}}{\beta^2\text{Precision} + \text{Recall}}.$$

- $\beta > 1$: emphasize recall.
- $0 < \beta < 1$: emphasize precision.

Changing the decision threshold

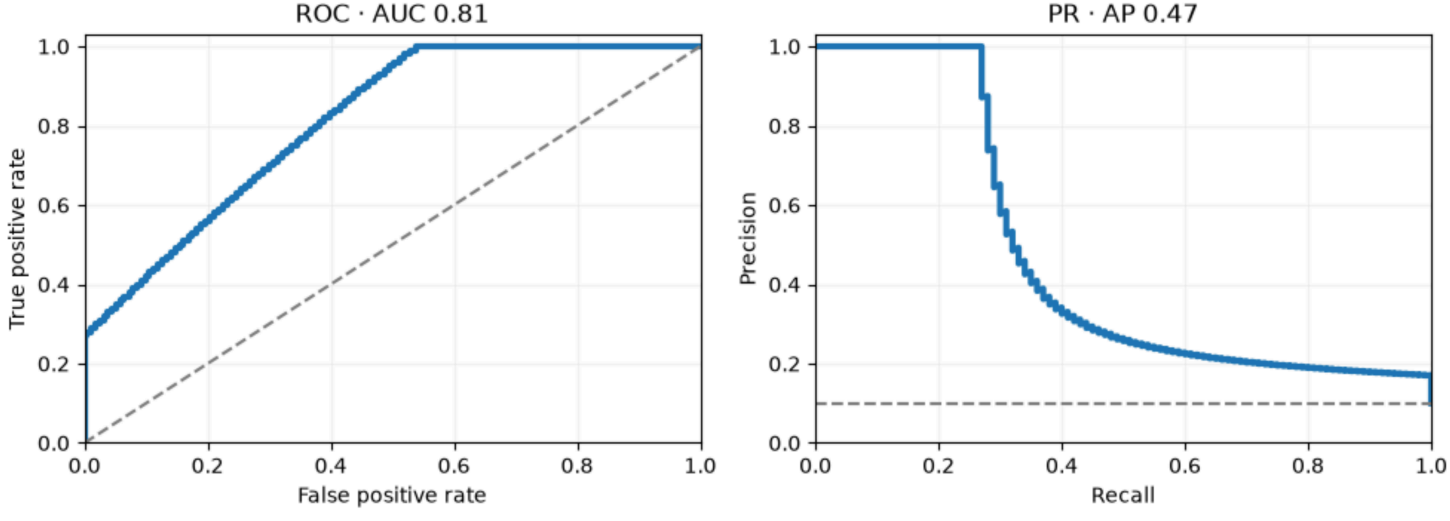
Decision threshold 0.5



TP: 65 · **FN:** 35 · **FP:** 240 · **TN:** 660 | **Precision:** 21.3% · **Recall:** 65.0%

Illustrative score distributions, not the same predictions as the fixed confusion matrix.

ROC and precision–recall curves



ROC-AUC summarizes ranking, not probability calibration. **Precision–recall** is particularly informative when positives are rare; its baseline equals positive-class prevalence (10% here).

Multiclass classification

For C classes, compute metrics per class and aggregate:

$$F_{1,\text{macro}} = \frac{1}{C}\sum_{c=1}^{C} F_{1,c},$$

$$F_{1,\text{weighted}} = \sum_{c=1}^{C} \frac{n_c}{N} F_{1,c}.$$

- **Macro:** each class receives equal weight.
- **Weighted:** each class receives weight proportional to support.
- **Micro:** aggregate TP, FP and FN before computing metrics.

In single-label multiclass classification, micro-F1 equals accuracy.

Regression metrics

$$\mathrm{MAE} = \frac{1}{N}\sum_i |y_i - \hat{y}_i|,$$

$$\mathrm{MSE} = \frac{1}{N}\sum_i (y_i - \hat{y}_i)^2,$$

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}},$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

- RMSE penalizes large errors more than MAE.
- R^2 compares against the mean-prediction baseline and may be negative.

Choosing metrics for the task

Application	Useful measures
Balanced classification	Accuracy, macro-F1
Rare-class screening	Recall, precision, PR-AUC
Fraud detection	Precision, recall, expected cost
Ranking	ROC-AUC, average precision
Reliable probabilities	Log loss, Brier score, calibration
Regression	MAE, RMSE, R^2

Select the evaluation metric before looking at the results. Otherwise, the metric itself becomes another tuning choice.

A complete evaluation workflow

1. Split off an **independent test set**.
2. Put standardization and the classifier in one **pipeline**.
3. Perform **stratified five-fold CV** on development data.
4. Tune with **GridSearchCV**.
5. Select the best configuration by mean CV accuracy.
6. **Refit** a new model on all development data.
7. Evaluate once on the test set using appropriate metrics.

The next slide presents this protocol using a built-in scikit-learn dataset.

Model selection: complete example

Run GridSearchCV + refit + test

Dataset: 569 breast cancer examples with 30 measured features.

- Reserve 20% as an independent test set.
- Tune the SVM's C and γ using stratified five-fold cross-validation.
- Refit on development data and evaluate once on the test set.

In the browser slide deck, click the button to run the example and see its results.

Take-home messages

- **Training error is not generalization error.**
- **Validation** guides choices, including early stopping.
- **K-fold CV** evaluates configurations using temporary models.
- **Refit** trains the final model on all development data.
- **Test** remains independent of model selection.
- **Metrics** should reflect the application and the costs of errors.
- Even a correctly evaluated model can fail under **distribution shift**.

Our objective is not merely the highest score, but a reliable procedure for model selection and performance estimation.